

then shifts more to smaller refinements in the solution. For example, in the traveling salesman problem with the Mississippi River twist, if λ is large, a decision to cross the Mississippi only twice is made at high T , while the specific routes on each side of the river are determined only at later stages.

The analogies to thermodynamics may be pursued to a greater extent than we have done here. Quantities analogous to specific heat and entropy may be defined, and these can be useful in monitoring the progress of the algorithm toward an acceptable solution [1].

CITED REFERENCES AND FURTHER READING:

- Salamon, P., Sibani, P., and Frost, R. 2002, *Facts, Conjectures, and Improvements for Simulated Annealing* (New York: SIAM Press).
- van Laarhoven, P.J.M., and Aarts, E.H.L. 1987, *Simulated Annealing: Theory and Applications* (Berlin: Springer).
- Kirkpatrick, S., Gelatt, C.D., and Vecchi, M.P. 1983, "Optimization by Simulated Annealing," *Science*, vol. 220, pp. 671–680.[1]
- Kirkpatrick, S. 1984, "Optimization by Simulated Annealing: Quantitative Studies," *Journal of Statistical Physics*, vol. 34, pp. 975–986.[2]
- Vecchi, M.P. and Kirkpatrick, S. 1983, "Global Wiring by Simulated Annealing," *IEEE Transactions on Computer Aided Design*, vol. CAD-2, pp. 215–222.[3]
- Otten, R.H.J.M., and van Ginneken, L.P.P.P. 1989, *The Annealing Algorithm* (Boston: Kluwer) [contains many references to the literature].[4]
- Metropolis, N., Rosenbluth, A., Rosenbluth, M., Teller A., and Teller, E. 1953, "Equations of State Calculations by Fast Computing Machines," *Journal of Chemical Physics*, vol. 21, pp. 1087–1092.[5]
- Lin, S. 1965, "Computer Solutions of the Traveling Salesman Problem," *Bell System Technical Journal*, vol. 44, pp. 2245–2269.[6]
- Numerical Recipes Software 2007, "Code Implementation for the Traveling Salesman Problem," *Numerical Recipes Webnote No. 14*, at <http://www.nr.com/webnotes?14> [7]
- Vanderbilt, D., and Louie, S.G. 1984, "A Monte Carlo Simulated Annealing Approach to Optimization over Continuous Variables," *Journal of Computational Physics*, vol. 56, pp. 259–271.[8]
- Numerical Recipes Software 2007, "Code for Minimization with Simulated Annealing," *Numerical Recipes Webnote No. 15*, at <http://www.nr.com/webnotes?15> [9]
- Bohachevsky, I.O., Johnson, M.E., and Stein, M.L. 1986, "Generalized Simulated Annealing for Function Optimization," *Technometrics*, vol. 28, pp. 209–217.[10]
- Corana, A., Marchesi, M., Martini, C., and Ridella, S. 1987, "Minimizing Multimodal Functions of Continuous Variables with the Simulated Annealing Algorithm," *ACM Transactions on Mathematical Software*, vol. 13, pp. 262–280.[11]
- Bélisle, C.J.P., Romeijn, H.E., and Smith, R.L. 1990, "Hide and Seek: A Simulated Annealing Algorithm for Global Optimization," Technical Report 90–25, Department of Industrial and Operations Engineering, University of Michigan.[12]
- Christofides, N., Mingozi, A., Toth, P., and Sandi, C. (eds.) 1979, *Combinatorial Optimization* (London and New York: Wiley-Interscience) [not simulated annealing, but other topics and algorithms].

10.13 Dynamic Programming

Dynamic programming, or *DP*, is an optimization technique that applies when a known sequence of choices, each with a cost or benefit, is to be made and one wants

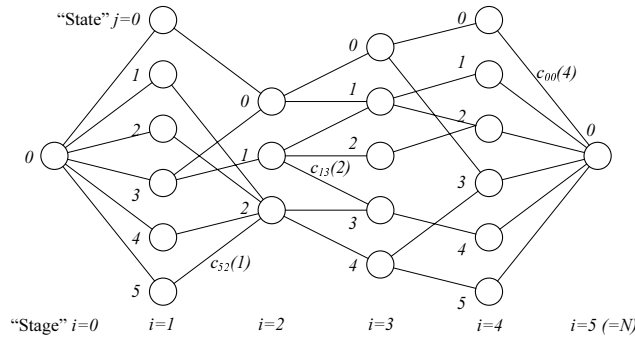


Figure 10.13.1. Canonical dynamical programming problem. It is desired to find the lowest cost path from a starting state to an ending state through $N - 1$ intermediate stages. Each stage is characterized by a set of *states* (not necessarily the same at each stage). An allowed edge between state j in stage i and state k in stage $i + 1$ has a cost denoted $c_{jk}(i)$ (not all labeled in the figure).

to minimize the total cost, or maximize the total benefit, after the sequence has been traversed. More specifically, a problem that is amenable to dynamic programming can be broken up into an ordered series of discrete *stages*, and within each stage a set of discrete *states*. These stages and states form a directed graph (see Figure 10.13.1) that we want to traverse from a given starting state ($i = 0$) to a given ending state ($i = N$). Allowed decisions that take one from state j in stage i to state k in stage $i + 1$ are edges in the graph. Their cost is denoted $c_{jk}(i)$. Without any loss of generality, one can connect all the states in stage i to all the states in stage $i + 1$, but with $c_{jk}(i) = \infty$ for forbidden paths.

Computer science is rich in graph-theoretic problems and algorithms, but only a few of these are within the scope of this book. Dynamic programming is one of these because its basic idea is very simple and its applications are very broad. It is important that you be able to recognize a problem amenable to DP when you see one. In particular, we will use several of the concepts in this section later, in §16.2, when we discuss the estimation of states from probabilistic data, including probabilistic decoding algorithms.

The key idea of dynamic programming is called the Bellman, Dijkstra, or Viterbi algorithm, depending on the field of training of the caller. As shown in Figure 10.13.2, the idea is that one can do a single sweep of a stage-ordered graph from left to right, labeling each vertex by the single number that is the cost of the *best* way of having reached it. (Henceforth we'll take the canonical DP problem as a cost-minimization problem; if your problem is instead a benefit-maximization problem, just use the negative of your benefits as costs.)

When the end state is reached, the global minimum cost of getting to it becomes known. Now, in a single backward pass, we can read off exactly what set of decisions led to this global minimum, by reconstructing which predecessor state was the one actually in the chain that led to the best result. Arriving back at the starting state, our solution is complete.

The art of DP involves, in many cases, the clever organization of the problem to minimize the number of states at each stage, so as to avoid the “curse of dimensionality” (a phrase first used by Bellman in exactly this context). Sometimes the order of the stages is not chronological at all, but merely reflects the decomposition of a

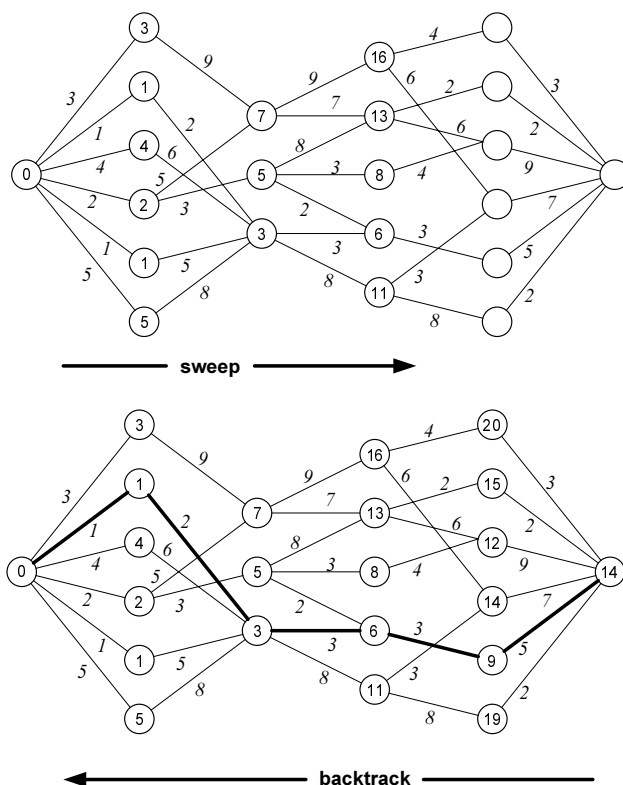


Figure 10.13.2. Two snapshots during the solution of a DP problem by the Bellman-Dijkstra-Viterbi algorithm. Edge costs are given as shown. Top: During the rightward sweep (here not yet complete) each state is labeled by the minimum cost to reach it, as determined solely by the labels of the preceding stage and the connecting edge costs. Bottom: After the rightward sweep is complete, the unique set of edges that produce the global minimum is found by one pass of backtracking.

problem into a convenient form for DP.

Here is a function embodying the Bellman-Dijkstra-Viterbi algorithm. You should consider this function more a precise statement of the algorithm than a production DP code. For example, it simply pushes off to the user function `cost()` the important issue of how to retrieve efficiently the (usually) sparse set of allowed edges that have finite costs. (You might want to consider a hash memory. See §7.6.) Also, this routine loops explicitly over all combinations of states j and k , the origin and destination states in going from stage i to $i + 1$. If you have a problem big enough to need some kind of sparse lookup, then you'll want to change these explicit loops accordingly.

```
VecInt dynpro(const VecInt &nstate,
Doub cost(Int jj, Int kk, Int ii)) {
    Given the vector nstate whose integer values are the number of states in each stage (1
    for the first and last stages), and given a function cost(j,k,i) that returns the cost of
    moving between state j of stage i and state k of stage i+1, this routine returns a vector
    of the same length as nstate containing the state numbers of the lowest cost path. States
    number from 0, and the first and last components of the returned vector will thus always
    be 0.
    const Doub BIG = 1.e99;
```

`dynpro.h`

```

static const Doub EPS=numeric_limits<Doub>::epsilon();
Int i, j ,k, nstage = nstate.size() - 1;
Doub a,b;
VecInt answer(nstage+1);
if (nstate[0] != 1 || nstate[nstage] != 1)
    throw("One state allowed in first and last stages.");
Doub **best = new Doub*[nstage+1];           Allocate array-of-arrays for storing scores.
best[0] = new Doub[nstate[0]];
best[0][0] = 0.;
for (i=1; i<=nstage; i++) {                  Forward sweep through stages.
    best[i] = new Doub[nstate[i]];
    for (k=0; k<nstate[i]; k++) {
        b = BIG;
        for (j=0; j<nstate[i-1]; j++) {      Find predecessor giving min cost.
            if ((a = best[i-1][j] + cost(j,k,i-1)) < b) b = a;
        }
        best[i][k] = b;
    }
}
answer[nstage] = answer[0] = 0;
for (i=nstage-1; i>0; i--) {                 Backtracking pass.
    k = answer[i+1];
    b = best[i+1][k];
    for (j=0; j<nstate[i]; j++) {            Find a predecessor that gave min.
        Doub temp = best[i][j] + cost(j,k,i);
        if (fabs(b - temp) <= EPS*fabs(temp)) break;
    }
    answer[i] = j;
}
for (i=nstage; i>=0; i--) delete [] best[i];  Cleanup storage.
delete [] best;
return answer;
}

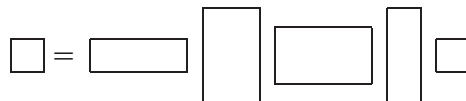
```

10.13.1 Example: Order of Matrix Multiplication

Suppose we have five matrices to multiply, so as to get a result T ,

$$T = ABCDE \quad (10.13.1)$$

The matrices may all have different shapes, as long as the number of columns of a matrix is the same as the number of rows of the matrix immediately to its right. Matrix multiplication is associative, and we can do the multiplies in any order we want; but the total number of scalar multiplications can be quite different, depending on which order is chosen. You should be able to see this in the following figure:



What we want to minimize is the total number of scalar multiplications. In this example, a good choice of “stage” is just how many matrix multiplications have been performed. So the stages and states might look something like this:

Stage 0	Stage 1	Stage 2	Stage 3	Stage 4
ABCDE	(AB)CDE A(BC)DE AB(CD)E ABC(DE)	(ABC)DE (AB)(CD)E (AB)C(DE) A(BCD)E A(BC)(DE) AB(CDE)	(ABCD)E (ABC)(DE) (AB)(CDE) A(BCDE)	(ABCDE)

Here parentheses group matrix factors that are already fully multiplied (i.e., they are, by now, a single matrix). We will leave it to you to connect the states by allowed edges, and to calculate the cost of each edge in terms of the dimensions of the various matrices.

So how could we have done this example *wrong*? We might have identified states with all possible ways of parenthesizing **ABCDE**, including, for example, **A(B(CD)E)**. That is unnecessary, because only the outermost parentheses matter: A matrix state doesn't care about the exact path taken to reach it, as long as its factors were multiplied in some associative order. The power of DP is realized when, at every stage, many histories collapse to a (relatively) small number of states, which can then be taken as ignorant of their past history.

10.13.2 Example: DNA Sequence Alignment

DNA sequences of different organisms, at one time identical in a common ancestor, can diverge over time by the deletion, insertion, or substitution of bases in one or the other organism's sequence. It is desired to find the best match between two given sequences. In finding the best match, we are allowed to insert gaps in either sequence; but in the end we will be assessed a penalty for gap positions, a penalty for mismatches, and a reward for matches.

For example [2], before matching, we might have the two sequences

```

G  A  A  T  T  C  A  G  T  T  A
G  G  A  T  C  G  A

```

A possible match might be

```

G  A  A  T  T  C  A  G  T  T  A
G  G  A      T  C      G      A

```

for which we would earn six rewards, less one mismatch penalty (shown *italic*) and four gap penalties. (We will consider all rewards and penalties as having positive or zero values, with the accounting done by *adding* rewards and *subtracting* penalties.)

Needleman and Wunsch [1] first pointed out that this problem is amenable to solution by DP, allowing *all possible* matchings to be scored, and the highest-scoring one identified. The clever idea is to form a two-dimensional array with the two sequences defining the columns and rows. In the above example, this looks like

		G	A	A	T	T	C	A	G	T	T	A
	0											
G												
G												
A												
T												
C												
G												
A												

A matching consists of a path through the (initially) empty boxes in the above tableau, starting in the box labeled by a zero and moving, in each step, either one box right, one box down, or diagonally down and right. A right or down move corresponds to using up a letter in the first or second sequence (respectively) without using up one in the second or first (respective) sequence. Therefore it corresponds to inserting a gap and incurs a gap penalty. A diagonal move corresponds to pairing a new character in each sequence. Therefore, it incurs either the matching reward, if the two sequences match, or a mismatch penalty, if they don't.

Also useful is to distinguish right or down moves between two boxes in the bordering rows or columns (respectively) of the table from right or down moves in the interior of the table. The former kind don't open gaps in either sequence, but merely allow the sequences to shift with respect to one another. So we might assess a smaller, or zero, penalty for these overall "skews."

Now, characteristic of dynamic programming, you just fill in the boxes with the total score of the *best* way to reach that box, either from above it, from its left, or from the upper-left diagonal neighbor. A score is computed, naturally, by taking the score of a predecessor box and then adding the reward (or subtracting the penalty) associated with the move. Starting at the upper left, there are always boxes ready to be filled in as you work your way, by rows or columns, to the lower right.

Also characteristic of dynamic programming, when you are done filling in all the boxes, you do a backtrack pass: Start at the lower right box in the table. Now figure out the path back through the table that contributed to that best score. (It may not be unique.) Finally, translate that path into the series of letters and gaps that it implies.

A straightforward implementation of this is the following routine. Note that the matching reward is normalized to one per matched character, while the penalties for mismatches, gaps, and overall skews are input arguments. You can set all three to zero in most cases. If you set a nonzero mismatch penalty, however, you will probably also want to have a gap penalty, since otherwise the program will always avoid a mismatch by creating two gaps, one in each string.

The routine's output includes a summary string showing where the matches, mismatches, and gaps occur. For the example above, the summary string is

= != == = =

(You can change the symbols used to your liking.)

You can modify the program in various other ways. For example, you might want to have a bigger penalty for initially opening a gap than for extending it once

opened. This requires a more complicated logic in the initial filling of the cost table. As long as you are able to fill each box with the cost of the *best* way to reach it, then the logic of dynamic programming will still apply.

```
void stringalign(char *ain, char *bin, Doub mispen, Doub gappen,          stringalign.h
                Doub skwpen, char *aout, char *bout, char *summary) {
    Given null terminated input strings ain and bin, and given penalties mispen, gappen, and
    skwpen, respectively, for mismatches, interior gaps, and gaps before/after either string, set null
    terminated output strings aout, bout, and summary as the aligned versions of the input strings,
    and a summary string. User must supply storage for the output strings of size equal to the sum
    of the two input strings.
    Int i,j,k;
    Doub dn,rt,dg;                                                    Cost of down, right, and diagonal moves.
    Int ia = strlen(ain), ib = strlen(bin);
    MatDoub cost(ia+1,ib+1);                                           Cost table, as illustrated in the text.
    First we fill in the cost table.
    cost[0][0] = 0.;
    for (i=1;i<=ia;i++) cost[i][0] = cost[i-1][0] + skwpen;
    for (i=1;i<=ib;i++) cost[0][i] = cost[0][i-1] + skwpen;
    for (i=1;i<=ia;i++) for (j=1;j<=ib;j++) {
        dn = cost[i-1][j] + ((j == ib)? skwpen : gappen);
        rt = cost[i][j-1] + ((i == ia)? skwpen : gappen);
        dg = cost[i-1][j-1] + ((ain[i-1] == bin[j-1])? -1. : mispen);
        cost[i][j] = MIN(MIN(dn,rt),dg);
    }
    Next, we do the backtrack pass, writing the output (backward, however).
    i=ia; j=ib; k=0;
    while (i > 0 || j > 0) {
        dn = rt = dg = 9.99e99;                                         Any large value will do.
        if (i>0) dn = cost[i-1][j] + ((j==ib)? skwpen : gappen);
        if (j>0) rt = cost[i][j-1] + ((i==ia)? skwpen : gappen);
        if (i>0 && j>0) dg = cost[i-1][j-1] +
            ((ain[i-1] == bin[j-1])? -1. : mispen);
        if (dg <= MIN(dn,rt)) {                                         Diagonal move produces either match or in-
            aout[k] = ain[i-1];                                           equality.
            bout[k] = bin[j-1];
            summary[k++] = ((ain[i-1] == bin[j-1])? '=' : '!');
            i--; j--;
        }
        else if (dn < rt) {                                             Down move produces a gap in the B string.
            aout[k] = ain[i-1];
            bout[k] = ' ';
            summary[k++] = ' ';
            i--;
        }
        else {                                                         Right move produces a gap in the A string.
            aout[k] = ' ';
            bout[k] = bin[j-1];
            summary[k++] = ' ';
            j--;
        }
    }
    Finally, reverse the output strings.
    for (i=0;i<k/2;i++) {
        SWAP(aout[i],aout[k-1-i]);
        SWAP(bout[i],bout[k-1-i]);
        SWAP(summary[i],summary[k-1-i]);
    }
    aout[k] = bout[k] = summary[k] = 0;                                Don't forget the terminating nulls!
}
```

Various modifications of the Needleman-Wunsch method are also in use, most

notably the generalization by Smith and Waterman [3]. There are also a number of heuristic methods for identifying sequence similarity, with names like BLAST, FASTA, BLAT, etc. The field is highly developed, so you should use the routine above only pedagogically.

CITED REFERENCES AND FURTHER READING:

- Cybenko, G. 1997, "Dynamic Programming: A Discrete Calculus of Variations," *IEEE Computational Science and Engineering*, vol. 4, no. 1, pp. 92-97.
- Bertsekas, 2001, *Dynamic Programming and Optimal Control*, 2nd ed., 2 vols. (Belmont, MA: Athena Scientific).
- Hillier, F.S. and Lieberman, G.J. 2002, *Introduction to Operations Research*, 7th ed. (New York: McGraw-Hill).
- Needleman, S.B. and Wunsch, C.D. 1970, "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins," *Journal of Molecular Biology*, vol. 48, pp. 443-453.[1]
- Rouchka, E.C. 2001, "Dynamic Programming," at multiple Web sites.[2]
- Smith, T.F. and Waterman, M.S. 1981, "Identification of Common Molecular Subsequences," *Journal of Molecular Biology*, vol. 147, pp. 195-197.[3]